Aim

To study the Solidity program structure, basic constructs, compilation, and deployment environment.

Theory

1. Introduction to Solidity

Solidity is a high-level, statically typed programming language used to develop smart contracts on Ethereum and other compatible blockchain platforms. Smart contracts are programs deployed on a blockchain that execute predefined instructions when specific conditions are met.

Solidity allows developers to create decentralized applications (DApps), manage digital assets, and implement business logic securely and transparently.

2. Solidity Program Structure

A Solidity program generally consists of the following components:

- **SPDX License Identifier:** Specifies the license under which the source code is distributed.
- **Pragma Directive:** Specifies the Solidity compiler version required to compile the program.
- **Contract Declaration:** Defines the smart contract using the `contract` keyword.
- **State Variables:** Store data permanently on the blockchain.
- **Constructor:** Initializes contract variables during deployment.
- **Functions:** Define the operations that the contract can perform.
- **Events:** Emit information about important contract activities for external applications.

A basic Solidity program follows this structure:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleContract {
    uint public number;

    constructor() {
        number = 0;
    }
```

```
    function setNumber(uint _number) public {
        number = _number;
    }

    function getNumber() public view returns (uint) {
        return number;
    }
}
```

3. Basic Constructs of Solidity

Primitive Data Types: Solidity supports several data types, including `uint` and `int` for integers, `bool` for Boolean values, `address` for Ethereum addresses, and `bytes` and `string` for binary and textual data.

Variables: Variables are classified into state variables, local variables, and global variables. State variables store data on the blockchain, local variables exist during function execution, and global variables provide information such as `msg.sender`, `msg.value`, and `block.timestamp`.

Functions: Functions define the operations performed by a smart contract. The `view` keyword allows a function to read state without modifying it, while `pure` functions neither read nor modify blockchain state.

Function Visibility: Solidity provides four visibility levels: `public`, `private`, `internal`, and `external`. These determine where functions and state variables can be accessed.

Constructors: A constructor executes once when a contract is deployed. It is commonly used to initialize state variables or assign the contract owner.

Modifiers: Modifiers allow developers to apply reusable conditions to functions, such as restricting access to authorized users.

Control Flow: Conditional statements such as `if-else` and loops such as `for` and `while` control the execution of contract logic.

Data Structures: Solidity supports arrays, mappings, structures (`struct`), and enumerations (`enum`) to organize and manage data efficiently.

Data Locations: The three main data locations are `storage`, `memory`, and `calldata`. They determine how data is stored and accessed during contract execution.

4. Compilation of Solidity Programs

Compilation converts Solidity source code into a format that the Ethereum Virtual Machine (EVM) can execute. The Solidity compiler checks the code for syntax errors and generates bytecode and an Application Binary Interface (ABI).

The ABI describes how external applications can interact with the compiled contract.

5. Deployment Environment

A smart contract can be deployed using development environments such as Remix IDE. Remix provides an editor, compiler, deployment interface, and tools to interact with smart contracts.

The Remix VM provides a simulated blockchain environment in which contracts can be deployed and tested without using real Ether. For public blockchain deployment, a compatible wallet and network configuration are required.

6. Transactions, Ether, Wei, and Gas

Ether is the native currency of Ethereum, and Wei is its smallest unit. One Ether equals (10^{18}) Wei.

Gas measures the computational resources required to execute operations on the blockchain. Transactions that modify contract state generally require gas, whereas read-only calls can typically be executed without submitting an on-chain transaction.

Understanding transactions and gas is essential for developing efficient smart contracts.

Implementation

The experiment was performed using Remix IDE to understand Solidity programming, compilation, deployment, and contract interaction.

Step 1: Open Remix IDE

1. Open Remix IDE in a web browser.
2. Create a new Solidity file with the `.sol` extension.
3. Write the Solidity program using the appropriate compiler version.

Step 2: Write the Solidity Program

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleContract {
    uint public number;
```

```
    constructor() {
        number = 0;
    }

    function setNumber(uint _number) public {
        number = _number;
    }

    function getNumber() public view returns (uint) {
        return number;
    }

    function increment() public {
        number++;
    }

    function decrement() public {
        require(number > 0, "Number cannot be negative");
        number--;
    }
}
```

Step 3: Compile the Contract

1. Open the Solidity Compiler panel.
2. Select a compatible compiler version.
3. Click **Compile SimpleContract.sol**.
4. Verify that the compilation completes without errors.

Step 4: Deploy the Contract

1. Open the Deploy & Run Transactions panel.
2. Select Remix VM as the execution environment.
3. Select `SimpleContract` from the contract dropdown.
4. Click **Deploy**.
5. Observe the deployed contract under the Deployed Contracts section.

Step 5: Interact with the Contract

1. Click `number` to retrieve the current value.
2. Enter a value in `setNumber` and execute the function.
3. Click `getNumber` to retrieve the updated value.
4. Execute `increment` to increase the stored value by one.
5. Execute `decrement` to decrease the stored value by one.

6. Observe the transaction results and updated state variable.

Step 6: Observe the Results

- The contract was compiled successfully.
- The contract was deployed in the Remix VM environment.
- The state variable was initialized to zero.
- The `setNumber` function updated the stored value.
- The `getNumber` function returned the current value.
- The `increment` and `decrement` functions modified the stored value.
- The `require` statement prevented the value from decreasing below zero.

Conclusion

Through this experiment, the structure and basic constructs of Solidity programming were studied and implemented. The program was compiled and deployed using Remix IDE, and its functions were tested in the Remix VM environment. The experiment provided practical knowledge of Solidity syntax, state variables, constructors, functions, and smart contract interaction. It also helped develop an understanding of the compilation and deployment process required for blockchain-based applications.